Тема 4. Алгоритмы сортировки и поиска на массивах

Лекция 6

Алгоритмы сортировки на массивах

Целью данной темы является изучение алгоритмов сортировки и поиска элементов массива. В ней осуществляется постановка задачи сортировки, формулируются свойства алгоритмов сортировки, рассматриваются базовые алгоритмы: сортировка обменом, вставками и выбором, а так же их различные улучшения, позволяющие существенно сократить их временную сложность. Так же, в этой теме осуществляется постановка задачи поиска и рассматриваются алгоритмы линейного, блочного и бинарного поиска. Проводится сравнительный анализ всех рассмотренных алгоритмов.

4.1. Постановка задачи сортировки

Пожалуй, никакая другая проблема не породила такого количества разнообразнейших решений, как задача сортировки. Приведем ее формальную постановку.

Задача сортировки. Пусть есть последовательность a_0, a_1... a_{n-1} и функция сравнения, которая на любых двух элементах последовательности принимает одно из трех значений: меньше, больше или равно. Требуется переставить члены последовательности таким образом, чтобы выполнялось условие: $a_i \leq a_{i+1}$, для всех i от 0 до $n - 1$.

Возможна ситуация, когда элементы состоят из нескольких полей, например, так, как показано на рис. 4.1.

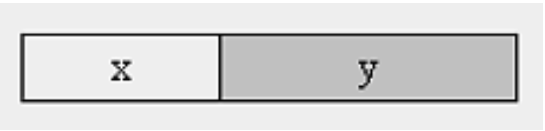


Рис. 4.1. Структура элемента последовательности

Если значение функции сравнения зависит только от поля *x*, то *x* называют ключом, по которому производится сортировка. На практике, в качестве *x* часто выступает число, а поле *y* хранит какие-либо данные, никак не влияющие на работу алгоритма.

Существует ли некий "универсальный", наилучший алгоритм? Вообще говоря, нет. Однако, имея приблизительные характеристики входных данных, можно подобрать метод, работающий оптимальным образом.

Ниже мы рассмотрим несколько алгоритмов решения задачи сортировки, и проведем их сравнительный анализ. Для того чтобы обоснованно сделать такой выбор, рассмотрим параметры, по которым будет производиться оценка алгоритмов.

1. *Временная сложность* сортировки – основной параметр, характеризующий быстродействие алгоритма.

2. *Память* – ряд алгоритмов требует выделения дополнительной памяти под временное хранение данных. При оценке используемой памяти не будет учитываться место, которое занимает исходный массив и независящие от входной последовательности затраты, например, на хранение кода программы.

3. *Устойчивость* – устойчивая сортировка не меняет взаимного расположения равных элементов. Такое свойство может быть очень полезным, если они состоят из нескольких полей, как на рис. 4.1, а сортировка происходит по одному из них, например, по *Key*. Свойство устойчивости удобно пояснить конкретным примером, представленным на рис. 4.2.



Рис. 4.2. Иллюстрация устойчивой и неустойчивой сортировки

В этом примере при устойчивой сортировке взаимное расположение равных элементов с ключом *1* и дополнительными полями "*a", "b", "c"* осталось прежним: элемент с полем *"a"*, затем – с *"b"*, затем – с *"c"*. При неустойчивой сортировке взаимное расположение равных элементов с ключом *1* и дополнительными полями "*a", "b", "c"* изменилось.

4. *Естественность поведения* – эффективность метода при обработке уже отсортированных, или частично отсортированных данных. Алгоритм ведет себя естественно, если учитывает эту характеристику входной последовательности и работает лучше.

Еще одним важным свойством алгоритма является его сфера применения. Здесь выделяют внутренние и внешние сортировки. *Внутренние сортировки* работают с данным в оперативной памяти с произвольным доступом, а *внешние сортировки* упорядочивают информацию, расположенную на внешних носителях. В настоящем разделе мы будем рассматривать только внутренние сортировки последовательностей, представленных массивами.

4.2. Элементарные методы сортировок

Метод сортировки обменом (метод пузырька). Расположим массив сверху вниз, от нулевого элемента – к последнему. Идея метода заключается в следующем. Шаг сортировки состоит в проходе снизу вверх по массиву. По пути просматриваются пары соседних элементов. Если элементы некоторой пары находятся в неправильном порядке, то меняем их местами. На рис. 4.3 приведен пример, иллюстрирующий применение этого метода для одного (нулевого) прохода. Пары сравниваемых элементов помечены серым цветом.

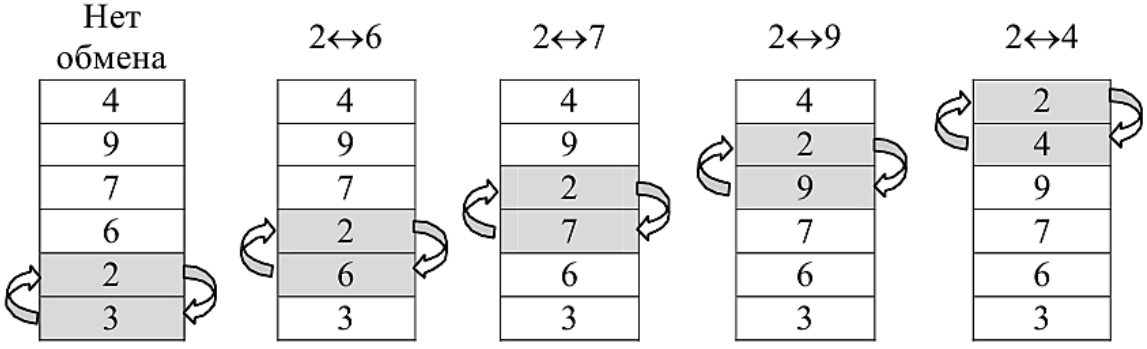


Рис. 4.3. Иллюстрация одного прохода сортировки обменом

После нулевого прохода по массиву "вверху" оказывается самый "легкий" элемент – отсюда аналогия с пузырьком. Следующий проход делается до второго сверху элемента, таким образом, второй по величине элемент поднимается на правильную позицию.

Делаем проходы по все уменьшающейся нижней части массива до тех пор, пока в ней не останется только один элемент. На этом сортировка заканчивается, так как последовательность упорядочена по возрастанию. На рис. 4.4 приведены результаты работы этого алгоритма. Отсортированная часть массива выделена серым цветом.

4	**2**	**2**	**2**	**2**	**2**
9	4	**3**	**3**	**3**	**3**
7	9	4	**4**	**4**	**4**
6	7	9	9	**6**	**6**
2	6	7	7	9	**7**
3	3	6	6	7	9
номер прохода	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$

Рис. 4.4. Результаты работы алгоритма сортировки обменом

Приведем блок-схему алгоритма сортировки обменом (алгоритм *BubbleSort*). Она представлена на рис. 4.5.

На практике метод пузырька, даже с улучшениями, работает, увы, слишком медленно. А потому почти не применяется.

Метод сортировки выбором. Идея метода состоит в том, чтобы создавать отсортированную последовательность путем присоединения к ней одного элемента за другим в правильном порядке. Следуя этой идее, будем строить готовую последовательность, начиная с левого конца массива. Алгоритм состоит из n-1 последовательных шагов, начиная от нулевого и заканчивая (n-2)-м. На i-м шаге выбираем наименьший из элементов a[i] ... a[n-1] и меняем его местами с a[i]. Последовательность шагов при n=5 изображена на рис. 4.6.

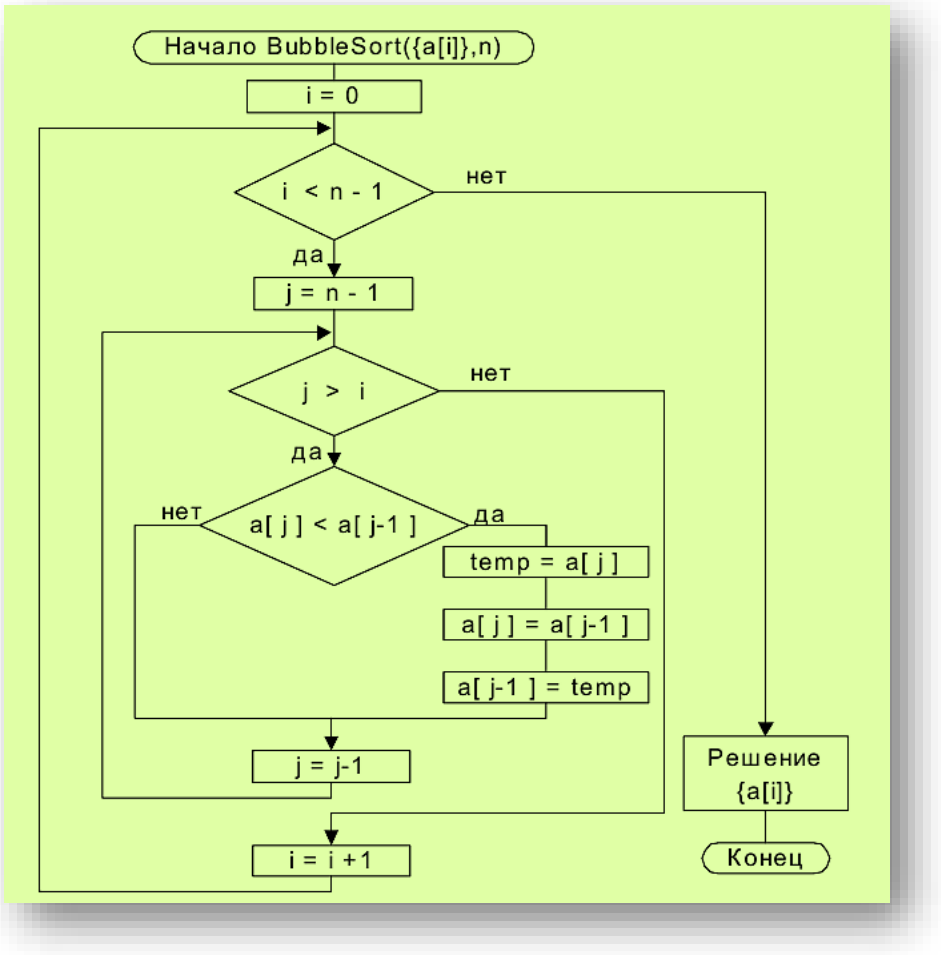


Рис. 4.5. Блок-схема алгоритма сортировки обменом

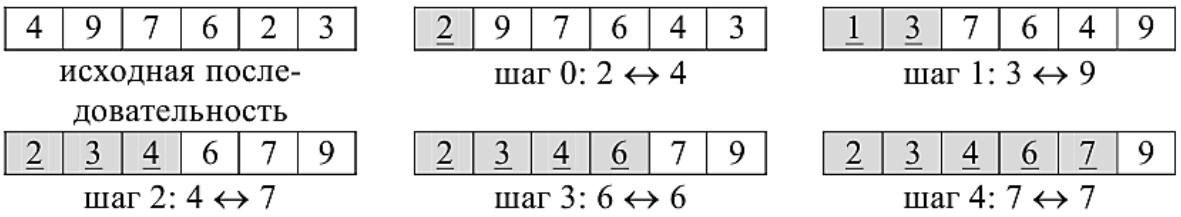


Рис. 4.6. Результаты работы алгоритма сортировки выбором

Вне зависимости от номера текущего шага i, последовательность *a[0]...a[i]* (на рис. 4.6 она выделена серым) является упорядоченной. Таким образом, на *(n-2)*-м шаге вся последовательность, кроме *a[n-1]* оказывается отсортированной, а *a[n-1]* стоит на последнем месте по праву: все меньшие

элементы уже ушли влево. На рис. 4.7 приведена блок-схема алгоритма сортировки выбором (*SelectSort*).

Метод сортировки вставками. Сортировка простыми вставками в чем-то похожа на вышеизложенные методы. Аналогичным образом делаются проходы по части массива, и аналогичным же образом в его начале "вырастает" отсортированная последовательность. Однако в сортировке пузырьком или выбором можно было четко заявить, что на *i*-м шаге элементы *a[0]...a[i]* стоят на правильных местах и никуда более не переместятся. Здесь же подобное утверждение будет более слабым: последовательность *a[0]...a[i]* упорядочена. При этом по ходу алгоритма в нее будут вставляться все новые элементы. Отсюда и название метода – вставками.

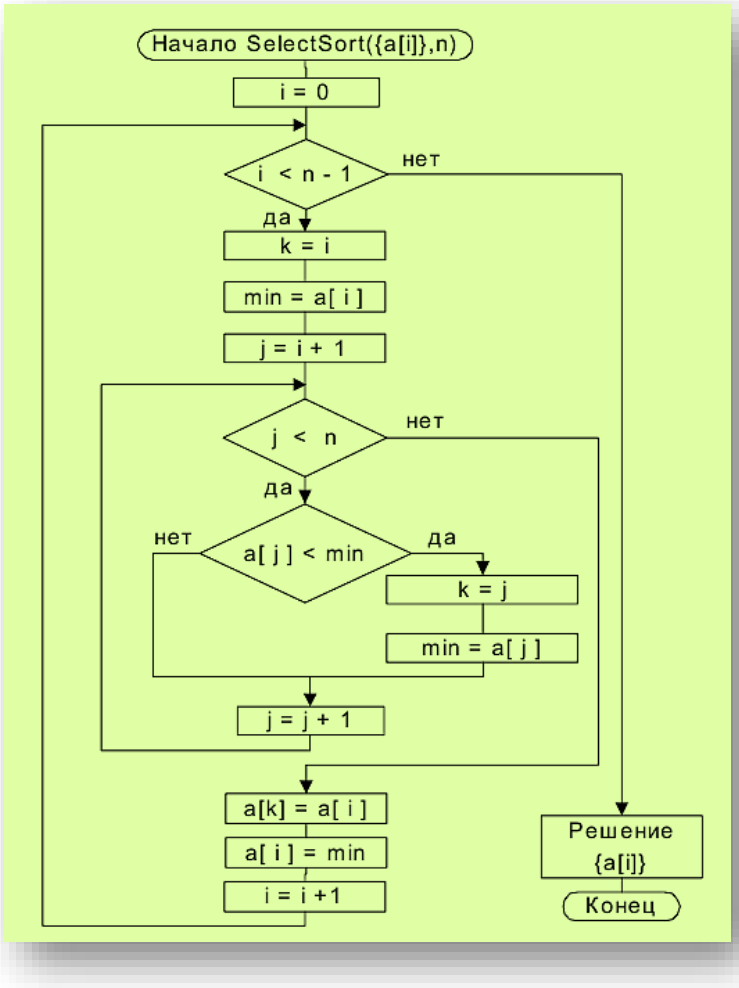


Рис. 4.7. Блок-схема алгоритма сортировки выбором

Будем разбирать алгоритм, рассматривая его действия на *i*-м шаге. Как говорилось выше, последовательность к этому моменту разделена на две части: готовую *a[0]...a[i]* и неупорядоченную *a[i+1]...a[n-1]*.

На следующем, *(i+1)*-м каждом шаге алгоритма берем *a[i+1]* и вставляем на нужное место в готовую часть массива. Поиск подходящего места для очередного элемента входной последовательности осуществляется путем последовательных сравнений с элементом, стоящим перед ним. В зависимости от результата сравнения элемент либо остается на текущем месте (вставка завершена), либо они меняются местами и процесс повторяется.

На рис. 4.8 приведен пример вставки очередного элемента из неупорядоченной последовательности в готовую. Здесь элементы *a[0]…a[3]* уже упорядочены (выделены серым цветом), а элемент *a[4]* вставляется в эту последовательность. Сравниваемые пары элементов выделены стрелками.

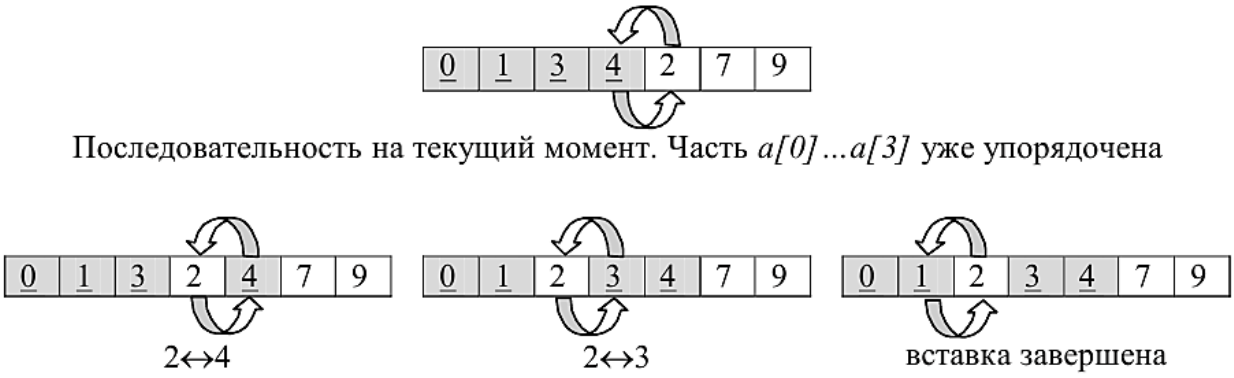


Рис. 4.8. Пример вставки очередного элемента в готовую последовательность

Таким образом, в процессе вставки мы "просеиваем" вставляемый элемент к началу массива, останавливаясь в случае, когда найден элемент, меньший, чем вставляемый или достигнуто начало последовательности. Такой процесс выполняется для каждого элемента массива.

На рис. 4.9 приведена блок-схема алгоритма сортировки вставками (*InsertSort*).

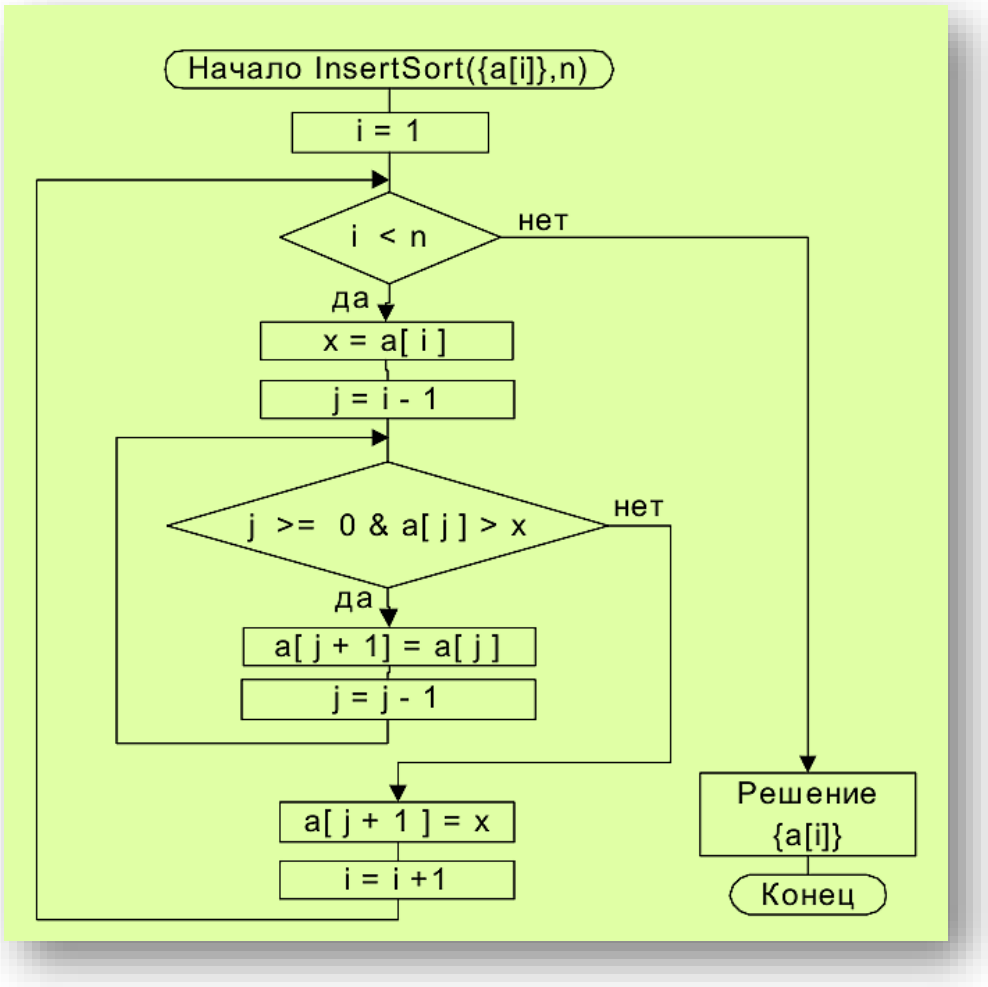


Рис. 4.9. Блок-схема алгоритма сортировки вставками

4.3. Анализ элементарных алгоритмов сортировок

Прежде чем проводить анализ рассмотренных ранее алгоритмов сделаем общие замечания к определению их временной сложности.

Во-первых, мерой объема входных данных служит количество элементов n, подлежащих сортировке.

Во-вторых, время работы программы сортировки может зависеть не только от количества сортируемых элементов, но и от того, насколько они упорядочены в исходной последовательности. Можно выделить два крайних

случая упорядоченности: когда сортируемый массив уже является упорядоченным как требуется, и когда массив упорядочен в обратной последовательности. Первый из них можно рассматривать как лучший случай исходных данных, т.к. фактически сортировка не требуется, а второй – как худший случай, поскольку сортировка на таких исходных данных практически приводит к полной перестановке элементов.

В-третьих, основными операциями сортировки являются сравнения элементов и их перестановка. Именно эти операции оказывают наибольшее влияние на время выполнения программы сортировки. Следовательно, временную сложность алгоритма сортировки можно определить подсчетом количества сравнений элементов и (или) их перестановок.

Анализ сортировки обменом. Во внутреннем цикле алгоритма BubbleSort выполняется одно сравнение и, в зависимости от результата этой операции либо три операции присваивания, обеспечивающие перестановку элементов, либо ни одной (см. рис. 4.5). Общее число сравнений элементов, выполняемое в алгоритме, не зависит от их начального порядка. Внутренний цикл алгоритма выполняет $n - i$ проход, где i – номер итерации внешнего цикла ($1 \le i \le n - 1$). Следовательно, общее число сравнений составляет

$$\sum_{i=1}^{n-1}(n-i) = n(n-1)/2 = n^2/2 - n/2 .$$

Перестановки в упорядоченном массиве не выполняются, а в массиве, упорядоченном в обратном порядке, их число равно числу сравнений элементов. Таким образом, количество операций присваивания равно

$$\begin{cases} 0, & \textit{в лучшем случае;} \\ 3 \cdot n^2/2 - 3 \cdot n/2, & \textit{в худшем случае.} \end{cases}$$

Исходя из этих подсчетов, можно утверждать, что временная сложность алгоритма *BubbleSort* имеет порядок $O(n^2)$. И, хотя общее время выполнения программы зависит от упорядоченности массива, нижняя временная граница является $\Omega(n^2)$. Таким образом, *BubbleSort* имеет временную сложность порядка $\Theta(n^2)$.

Для реализации алгоритма обменной сортировки дополнительная память не требуется: все операции происходят "на месте".

Поскольку мы определили, что временная сложность данного алгоритма практически не зависит от начальной упорядоченности массива, его поведение можно считать не естественным.

В этом виде сортировки обмен не затрагивает элементы с одинаковыми значениями ключа, и, следовательно, ранее стоящие равные элементы "всплывут" к началу последовательности раньше последующих. Т.е. их естественный порядок следования не изменится, следовательно, сортировка обменом устойчива.

Анализ сортировки выбором. Во внутреннем цикле алгоритма *SelecSort* (см. рис. 4.7) выполняется одно сравнение и, в зависимости от результата этой операции либо одна операции присваивания, предназначенная для сохранения текущего найденного минимума, либо ни одной. Во внешнем цикле алгоритма выполняются три операции присваивания, обеспечивающие перестановку элементов.

Общее число сравнений элементов, выполняемое в алгоритме, не зависит от их начального порядка. Внутренний цикл алгоритма выполняет $n - i$ проход, где i – номер итерации внешнего цикла ($1 \le i \le n - 1$). Следовательно, общее число сравнений составляет

$$\sum_{i=1}^{n-1}(n-i) = n(n-1)/2 = n^2/2 - n/2 .$$

Кроме этого во внешнем цикле, вне зависимости от начальной упорядоченности массива выполняются три операции присваивания. В упорядоченном массиве операция присваивания во внутреннем цикле не выполняется, а в массиве, упорядоченном в обратном порядке, количество таких операций совпадает с количеством сравнений. Таким образом, количество операций присваивания равно

$$\begin{cases} 3 \cdot n - 3, & \textit{в лучшем случае;} \\ n^2/2 + 5 \cdot n/2 - 3, & \textit{в худшем случае.} \end{cases}$$

Исходя из этих подсчетов, можно утверждать, что временная сложность алгоритма *SelectSort* имеет порядок $O(n^2)$. И, хотя общее время выполнения программы зависит от упорядоченности массива, нижняя временная граница является $\Omega(n^2)$. Таким образом, *SelectSort* имеет временную сложность порядка $\Theta(n^2)$.

Для реализации алгоритма сортировки выбором дополнительная память не требуется.

Поскольку мы определили, что временная сложность данного алгоритма практически не зависит от начальной упорядоченности массива, его поведение можно считать не естественным.

Сортировка выбором осуществляет обмен между первым неупорядоченным элементом и первым найденным минимальным, что, в общем случае может нарушить естественный порядок следования элементов. Это легко продемонстрировать следующим примером. Рассмотрим последовательность из трех элементов, каждый из которых имеет два поля, а сортировка идет по первому из них (см. рис. 4.10). Результат ее сортировки можно увидеть уже после шага 0, так как больше обменов не будет. Порядок ключей *2a, 2b* был изменен на *2b, 2a*, так что алгоритм сортировки выбором неустойчив.



Рис. 4.10. Иллюстрация неустойчивости сортировки выбором

Анализ сортировки вставками. Операция сравнения элементов в алгоритме *InsertSort* включена в условие продолжения внутреннего цикла. Одна операция присваивания выполняется во внутреннем цикле алгоритма и две во внешнем (см. рис. 4.9).

Если первоначальный массив отсортирован, то внутренний цикл алгоритма не выполняется, т.к. условие его продолжения сразу является ложным. Внешний цикл завершается за $n–1$ итерацию. Т.е. в лучшем случае в алгоритме выполняется $n – 1$ сравнение и $2n – 2$ присваивание.

Если массив первоначально отсортирован в обратном порядке, то внутренний цикл алгоритма выполняет по i итераций, где i – номер итерации внешнего цикла ($1 \le i \le n - 1$). Общее количество сравнений в этом случае равно

$$\sum_{i=1}^{n-1} i = n(n-1)/2 = n^2/2 - n/2.$$

Такое же число операций присваивания выполняется во внутреннем цикле в дополнение к $2n - 2$ операциям сравнения, выполняемых во внешнем цикле.

Таким образом, имеем:

$$\text{количество сравнений} = \begin{cases} n-1, & \textit{в лучшем случае;} \\ n^2/2 - n/2, & \textit{в худшем случае.} \end{cases}$$

$$\text{количество операций присваивания} = \begin{cases} 2 \cdot n - 2, & \textit{в лучшем случае;} \\ n^2/2 + 3 \cdot n/2 - 2, & \textit{в худшем случае.} \end{cases}$$

Исходя из этих подсчетов, можно утверждать, что временная сложность алгоритма *InsertSort* имеет верхний порядок роста $O(n^2)$ и нижний порядок роста $\Omega(n)$.

Для реализации сортировки вставками дополнительная память не требуется.

Хорошим показателем такой сортировки является весьма естественное поведение: почти отсортированный массив будет досортирован очень быстро.

Очевидно, что в сортировке вставками элементы с равными ключами продвигаются на свои места в отсортированной последовательности в естественном порядке. Таким образом, этот вид сортировки устойчив.

И так, мы определили характеристики алгоритмов сортировок обменом, выбором и вставками. Результаты проведенного анализа сведены в таблицу 4.1.

Таблица 4.1

Алгоритм	Временная сложность	Дополнительная память	Устойчивость	Естественность поведения
BubbleSort	$\Theta(n^2)$	**не требуется**	**да**	**нет**
SelectSort	$\Theta(n^2)$	**не требуется**	**нет**	**нет**
InsertSort	$\Omega(n)$, $O(n^2)$	**не требуется**	**да**	**да**